When Does More API Spend Stop Helping? Measuring and Exploiting Marginal Returns in LLM-Based Software Engineering

Research Area: Software Engineering, Artificial Intelligence, and Cost-Aware LLM Systems

Abstract

Application programming interface (API)-based large language models (LLMs) are increasingly used in software-engineering workflows, including code generation, bug fixing, test generation, code review, and repository-level question answering. Although existing research has proposed prompt caching, context compression, semantic caching, model routing, and turn limits to reduce API expenditure, these techniques generally optimize individual mechanisms rather than determine whether each additional unit of spending improves the probability of successfully completing a software-engineering task [1], [2], [3]. This paper proposes the Marginal Value of Spend (MVS) framework, which treats API expenditure as an economic investment in task resolution. MVS measures the marginal increase in task-success probability obtained from an additional API call, retrieval step, retry, context expansion, verification action, or model upgrade. Based on this measure, an LLM-based coding agent can decide whether to continue, stop, retrieve additional evidence, verify its output, or escalate to a stronger model [4], [5]. The paper introduces Marginal Success per Dollar (MSD) as a primary evaluation metric and proposes a developer-time-adjusted cost model that considers the cost of repairing incorrect or incomplete outputs [6]. The proposed framework will be evaluated on code-generation, bug-fixing, test-generation, code-review, and repository-question-answering tasks. The evaluation will compare MVS-based adaptive control with fixed-budget, fixed-turn, single-model, routing-only, caching-only, and compression-only baselines. The expected contribution is a cost-aware control policy that identifies the point at which additional API spending no longer produces economically meaningful returns, thereby improving the cost–quality frontier of LLM-based software-engineering systems [5], [7], [8].

Keywords

API cost optimization; LLM-based software engineering; marginal value of spend; cost-aware coding agents; marginal success per dollar; model routing; prompt caching; token reduction; cost–quality trade-off; developer productivity.

1. Introduction

Large language models are increasingly integrated into software-engineering tools through commercial APIs. Developers use these models to generate code, repair defects, write unit tests, review source-code changes, explain unfamiliar repositories, and automate multi-step engineering workflows. These capabilities can reduce manual effort and accelerate software development. However, they also introduce a recurring financial cost because every prompt, completion, agent action, retrieval operation, retry, and model escalation may consume billable tokens [7], [9].

The cost problem becomes more significant in repository-level software engineering. Unlike short single-turn prompts, repository-level tasks often require an agent to inspect multiple files, understand dependencies, run tests, read execution logs, generate patches, and revise its solution after receiving feedback. As the number of agent turns increases, both the number of API calls and the volume of context transferred to the model can grow rapidly. Prior studies have therefore investigated prompt caching, context management, prompt compression, semantic caching, and model routing as approaches for reducing LLM API cost [2], [3], [4], [10].

However, reducing the number of tokens or selecting a cheaper model does not necessarily reduce the total cost of software engineering. A low-cost model may generate an incorrect patch that requires substantial human repair. Similarly, additional retries or escalation to a stronger model may improve success rates, but the improvement may be too small to justify the additional API expenditure. Recent empirical work shows that token reduction is not always equivalent to end-to-end cost reduction; in one study, a compression configuration reduced delivered tool-output tokens by 38.4% but increased billed cost by 6.8% [5]. Therefore, cost optimization should focus on the total cost of obtaining an acceptable engineering outcome rather than only minimizing token consumption.

This paper addresses a gap in existing research: the lack of an explicit economic model for deciding when additional API spending is worthwhile in LLM-based software-engineering workflows. Instead of asking only “How can API cost be reduced?”, this research asks, “When does an additional unit of API spending produce a meaningful increase in the probability of successfully completing a task?” This concept is referred to as the marginal value of spend.

The proposed Marginal Value of Spend framework tracks the cumulative API cost of a task and estimates the marginal gain in success probability from further spending. The framework uses this estimate to make adaptive decisions: continue the current strategy, request additional retrieval, retry a failed verification, escalate to a stronger model, or terminate and return the best available output. The central decision rule is that additional

spending should continue only when its expected benefit exceeds its incremental cost and the risk of accepting an incorrect result.

The main contributions of this paper are as follows:

1. It introduces an economic perspective on API cost in LLM-based software engineering based on marginal returns rather than simple token minimization.
2. It defines Marginal Success per Dollar (MSD), a metric that measures the additional probability of task success obtained per unit of additional API expenditure.
3. It proposes an MVS-based adaptive control policy that helps an LLM coding agent decide when to stop, retry, retrieve more context, or escalate to a stronger model.
4. It presents an evaluation methodology for measuring spend–success relationships and comparing MVS with fixed-budget, fixed-turn, routing, caching, and compression baselines.

The remainder of this paper is organized as follows. Section 2 reviews related literature. Section 3 presents the problem formulation and the proposed MVS framework. Section 4 describes the proposed empirical evaluation methodology. Section 5 discusses expected outcomes and limitations. Section 6 concludes the paper and outlines future work.

2. Review of Literature

2.1 API Cost in LLM-Based Software Engineering

The financial cost of API-based LLM systems has become an important concern for organizations adopting AI-assisted software development. Unlike traditional machine-learning systems, where costs are often incurred mainly during training or local inference, API-based systems incur recurring costs for every request. Empirical studies of LLM application development identify API-call cost as a major challenge, especially for applications that require repeated calls, long contexts, or multi-step reasoning [7], [9].

In software engineering, API cost is amplified by agentic workflows. A coding agent may need to read a repository, identify relevant files, generate a patch, execute tests, inspect failures, and revise its solution. Each step may require additional prompts and completions. Therefore, the total cost of solving a task depends not only on the size of the final prompt but also on the number of agent actions, the amount of context transferred, and the frequency of retries or model escalations [5], [8].

Recent work on enterprise coding agents also shows that direct API expenditure cannot be considered separately from developer productivity. Lower API spending may lead to lower-quality outputs, which can increase the time developers spend reviewing, correcting, or rewriting AI-generated code [6]. This observation supports the need for a cost model that includes both API expenditure and human repair effort.

2.2 Prompt Caching and Semantic Caching

Prompt caching reduces API cost by reusing previously processed context across multiple requests. In coding agents, stable content such as system instructions, repository conventions, tool definitions, and project-level documentation can be placed at the beginning of a prompt so that it can be reused in later requests. Lumer et al. evaluated prompt caching across OpenAI, Anthropic, and Google models and found that it reduced API costs by 41–80% and improved time to first token by 13–31%. Their study also found that strategic cache-block placement was more effective than naïve full-context caching [2].

A production-oriented case study by Peng, Lin, and Lee examined enterprise coding agents and found that a 99.3% prompt-cache hit rate reduced realized API cost by 88.6%. The study also emphasized that lower direct API cost does not automatically imply lower total engineering cost because developer churn and output-quality issues can offset savings [6].

Semantic caching extends prompt caching by reusing responses for semantically similar requests rather than requiring exact prompt equality. Embedding-based semantic caching compares an incoming query with stored query embeddings and retrieves a prior response when the similarity is sufficiently high [9]. Although semantic caching can reduce repeated expenditure, it introduces risks in software engineering. Code may change between requests, dependencies may be updated, and a previously correct answer may become invalid after a repository modification. Therefore, cache invalidation policies should consider code differences, dependency versions, test outcomes, and model versions.

Completion caching has also been proposed in FrugalGPT, where responses to submitted queries are stored locally and reused when identical or sufficiently similar queries occur [3]. This approach is useful for repeated developer questions, but it is less effective for unique debugging tasks, where repository context and failure conditions often differ between requests.

2.3 Context Management and Token Reduction

Context management reduces API cost by limiting the amount of irrelevant information sent to the model. In repository-level tasks, sending entire files, complete chat histories, or large execution logs can be expensive and may also reduce performance by diluting task-relevant evidence. Research on prompt-centric LLM access identifies model selection, context management, and caching as three major opportunities for cost optimization [4].

Prompt compression is a related technique. It reduces the number of input tokens by removing redundant or low-information content while attempting to preserve information

required for the task. Cache-aware prompt-compression research combines compression with API caching and models the interaction between cached and uncached tokens [1]. However, aggressive compression can remove identifiers, type information, API signatures, or failure details that are essential for debugging and code repair.

Recent empirical work warns that token reduction should not be treated as equivalent to cost reduction. Weinberger and Hozez found only a weak correlation between token reduction and cost reduction and showed that compression can change agent trajectories through additional retrieval, diagnosis, testing, and turns, thereby offsetting local token savings [5]. This finding motivates the present research: the objective should not be to spend as little as possible, but to spend where additional context or reasoning is most likely to improve task success.

2.4 Model Routing, Cascading, and Turn Control

Model routing assigns each request to the least expensive model that is expected to satisfy a required quality threshold. Model cascading extends this idea by first using a cheaper model and escalating to a more capable model only when necessary. FrugalGPT is an influential example of this approach and demonstrates that cascades can reduce cost while maintaining performance [3]. More recent work on cost-aware model orchestration incorporates quality and cost trade-offs into model-selection decisions [10].

Turn control is another important cost lever in agentic software engineering. Coding agents may continue iterating even when additional actions are unlikely to improve the solution. An empirical study of turn-control strategies found that well-calibrated fixed-turn limits can reduce costs by up to 68% with minimal impact on solve rates, and may even improve performance by encouraging more focused behavior [8]. However, fixed limits are blunt instruments: they may terminate difficult tasks too early while allowing easy tasks to consume unnecessary budget.

The proposed MVS framework differs from fixed-turn control by making the stopping decision conditional on expected marginal benefit. Rather than applying the same turn limit to every task, MVS estimates whether another agent action is likely to produce a meaningful improvement relative to its cost.

2.5 Cost-Aware Evaluation of Coding Agents

Traditional software-engineering benchmarks often emphasize functional correctness, such as whether generated code passes tests or whether an agent resolves an issue. However, cost-aware evaluation requires additional measures, including token consumption, API expenditure, latency, cache hit rate, escalation rate, and the cost of human intervention. Resource-aware agent benchmarks have begun to measure correctness alongside latency, cost, compute, memory, and network usage under declared resource constraints [11].

End-to-end efficiency studies argue that evaluation should use success-adjusted billing rather than raw token counts, because a cheaper but unsuccessful attempt may ultimately be more expensive than a more expensive successful attempt [5]. Therefore, the proposed research uses cost per successfully resolved task as its primary evaluation metric.

Cost-aware evaluation should also include developer repair time. If a low-cost model produces incorrect code, the developer may need to spend additional time identifying and correcting the error. Therefore, the total cost of an AI-assisted software-engineering task includes API expenditure, verification cost, latency, and the expected cost of human repair [6].

2.6 Research Gap

Although prior studies have developed effective mechanisms for reducing API cost, they largely treat cost reduction as a local optimization problem. Caching reduces repeated context cost, compression reduces input tokens, routing selects cheaper models, and turn limits restrict agent iterations [2], [3], [4], [8], [10]. However, these methods do not provide a unified task-level economic policy for determining whether additional spending is justified at a particular point in a software-engineering workflow.

In particular, little prior work measures the marginal return of additional API expenditure across heterogeneous software-engineering tasks. It remains unclear whether an additional retrieval step, another test-driven repair cycle, a larger context window, or escalation to a frontier model provides sufficient improvement in success probability to justify its cost. The proposed MVS framework addresses this gap by measuring marginal success per dollar and using it to guide adaptive stopping, retrying, retrieval, and escalation decisions.

3. Problem Formulation and Proposed Framework

The MVS framework treats each software-engineering task as a sequential decision process. At each step, the agent has a current solution, a cumulative API cost, and an estimate of the probability that further spending will improve the outcome. The framework evaluates whether the expected benefit of another action exceeds its incremental cost.

Let s denote cumulative API spend and P(s) denote the estimated probability of successful task completion at spend level s. The marginal value of an additional spend increment Δs is defined as:

$$MV(\Delta s) = P(s + \Delta s) - P(s)$$

The expected net value of spending Δs is defined as:

$$EV(\Delta s) = MV(\Delta s) \times V_task - \Delta s - C_risk$$

where V_task represents the value of successfully completing the task, and C_risk represents the expected cost of accepting an incorrect, incomplete, or low-quality output. The agent continues spending only when $EV(\Delta s) > 0$.

Marginal Success per Dollar is defined as:

$$MSD = [P(s + \Delta s) - P(s)] / \Delta s$$

A high MSD indicates that additional spending is likely to be productive. A low or negative MSD suggests that the agent should stop, simplify its approach, or return its current best solution.

The MVS controller can take four actions:

- Continue: spend more on the current strategy when the expected marginal value is positive.
- Retrieve or expand context: spend on additional relevant evidence when the current context appears insufficient.
- Retry or verify: spend on additional testing, static analysis, or repair when the current output is uncertain.
- Stop or escalate: terminate when further spending is unlikely to help, or escalate to a stronger model when the expected benefit justifies the higher cost.

4. Proposed Empirical Evaluation Methodology

The proposed framework will be evaluated through a controlled empirical study. The study will measure how success probability changes as API spending increases across multiple software-engineering tasks and compare MVS-based control with existing cost-reduction strategies.

The evaluation will include five task categories: code generation, automated bug fixing, unit-test generation, code review, and repository-level question answering. Repository-level bug fixing is particularly important because it requires multi-step reasoning, retrieval, testing, and patch revision. Public benchmarks such as HumanEval and MBPP can be used for controlled code-generation evaluation, while SWE-bench-style repository tasks can be used for more realistic issue-resolution evaluation.

The primary dependent variable will be cost per successfully resolved task. Secondary measures will include total API cost, input and output token counts, cache hit rate, latency, task success rate, test-pass rate, escalation rate, and estimated developer repair time.

The study will compare the following baselines:

- Frontier-model-only baseline: use a high-capability model for every request.
- Low-cost-model-only baseline: use an inexpensive model for every request.
- Fixed-budget baseline: apply the same API budget to every task.
- Fixed-turn baseline: apply the same maximum number of agent turns to every task.
- Prompt-cache-only baseline: apply stable-prefix caching without adaptive control.
- Compression-only baseline: apply prompt compression without adaptive control.
- Routing-only baseline: use model cascading without MVS-based stopping.
- MVS framework: combine adaptive stopping, retrieval control, verification, and escalation based on marginal value of spend.

The main analysis will plot spend–success curves for each task type and model configuration. These curves will show whether returns diminish smoothly, whether certain tasks benefit from sustained spending, and whether additional spending on easy tasks is wasteful. Ablation studies will remove individual MVS components, such as verification, escalation, retrieval control, and adaptive stopping, to determine which components contribute most to cost efficiency.

Statistical analysis will use paired comparisons across tasks, bootstrap confidence intervals, and mixed-effects models to account for variation across repositories, issue difficulty, and model behavior. Because LLM outputs are stochastic, each configuration will be repeated multiple times where feasible.

5. Expected Outcomes and Limitations

The study is expected to show that API spending has diminishing marginal returns in LLM-based software-engineering workflows. Early spending may improve success probability substantially by providing relevant context, generating an initial solution, or performing basic verification. Later spending may produce smaller improvements because the agent may already have sufficient information or may be repeatedly attempting an unsuccessful strategy.

The study also expects to show that fixed budgets and fixed turn limits are suboptimal. A fixed budget may waste money on easy tasks and under-spend on difficult tasks. In contrast, MVS-based control is expected to allocate spending according to the estimated marginal benefit of further actions.

Several limitations must be considered. First, API prices and model capabilities can change over time, which may affect the generalizability of results. Second, LLM outputs are stochastic, so repeated runs are necessary to obtain reliable estimates. Third, benchmark contamination may affect the validity of code-generation results. Fourth,

estimating developer repair time may be difficult because it depends on developer expertise, project complexity, and organizational practices. These limitations will be addressed through transparent reporting, repeated trials, contamination checks, and sensitivity analysis.

6. Conclusion and Future Work

This paper proposes a shift from token-centric API cost reduction to spend-centric decision-making in LLM-based software engineering. Instead of treating every token, retry, retrieval step, or model upgrade as a cost to minimize, the Marginal Value of Spend framework asks whether each additional unit of expenditure is likely to improve the probability of successful task completion by enough to justify its cost.

The proposed work introduces Marginal Success per Dollar as an evaluation metric and an MVS-based adaptive control policy for coding agents. By measuring spend–success relationships across software-engineering tasks, the research can identify when additional API spending is productive, when it produces diminishing returns, and when it should be stopped.

Future work will validate the framework through real-world experiments, incorporate developer-time data, examine multi-model routing under changing API prices, and extend MVS to team-level software-engineering workflows where multiple agents, reviewers, and human developers interact.

References

[1] Cache-Aware Prompt Compression: A Two-Tier Cost Model for LLM API Caching, arXiv preprint arXiv:2607.15516, 2026. Available: https://arxiv.org/abs/2607.15516

[2] E. Lumer, F. Nizar, A. Jangiti, K. Frank, A. Gulati, M. Phadate, and V. K. Subbiah, “Don’t Break the Cache: An Evaluation of Prompt Caching for Long-Horizon Agentic Tasks,” arXiv preprint arXiv:2601.06007, 2026. doi: 10.48550/arXiv.2601.06007.

[3] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance,” arXiv preprint arXiv:2305.05176, 2023. Available: https://arxiv.org/abs/2305.05176

[4] Reducing Costs to Access LLMs in a Prompt-Centric Internet, arXiv preprint arXiv:2410.11857, 2024. Available: https://arxiv.org/abs/2410.11857

[5] S. Weinberger and A. Hozez, “Token Reduction Is Not Cost Reduction,” arXiv preprint arXiv:2607.12161, 2026. doi: 10.48550/arXiv.2607.12161.

[6] S.-W. Peng, Y.-H. Lin, and Y.-P. Lee, “Inference Economics of Enterprise Coding Agents: A Case Study of Cloud vs. On-Premise LLMs,” arXiv preprint arXiv:2607.13080, 2026. doi: 10.48550/arXiv.2607.13080.

[7] An Empirical Study on Challenges for LLM Application Developers, arXiv preprint arXiv:2408.05002, 2024. Available: https://arxiv.org/abs/2408.05002

[8] More with Less: An Empirical Study of Turn-Control Strategies for Efficient Coding Agents, arXiv preprint arXiv:2510.16786, 2025. Available: https://arxiv.org/abs/2510.16786

[9] Reducing LLM Costs and Latency via Semantic Embedding Caching, arXiv preprint arXiv:2411.05276, 2024. Available: https://arxiv.org/abs/2411.05276

[10] Cost-Aware Model Orchestration for LLM-based Systems, arXiv preprint arXiv:2512.01099, 2025. Available: https://arxiv.org/abs/2512.01099

[11] AgentSLABench: Evaluating and Benchmarking Agentic Systems Under Resource Constraints, arXiv preprint arXiv:2608.00805, 2026. Available: https://arxiv.org/abs/2608.00805